produces:

```
Caller likes 7
```

Within a raw proc, a break terminates the method that invoked the block. The return value of the method is any parameters passed to the break.

Return and Blocks

A return from inside a *raw block* that is inside a scope acts as a return from that scope. A return from a block whose original context is no longer valid raises an exception (LocalJumpError or ThreadError depending on the context). The following example illustrates the first case:

```
def meth1
  (1..10).each do |val|
    return val           # returns from meth1
  end
end

meth1     # => 1
```

The following example shows a return failing because the context of its block no longer exists:

reference/local_jump_error.rb

```
def meth2(&b)
  b
end

res = meth2 { return }
res.call
```

(produces no output)

And here's a return failing because the block is created in one thread and called in another:

```
def meth3
  yield
end

t = Thread.new do
meth3 { return }
end

t.join
```

produces:

```
#<Thread:0x0000000102428aa8 prog.rb:5 run> terminated with exception
(report_on_exception is true):
        from prog.rb:2:in `meth3'
        from prog.rb:6:in `block in <main>'
        from prog.rb:2:in `meth3'
        from prog.rb:6:in `block in <main>'
prog.rb:6:in `block (2 levels) in <main>': unexpected return (LocalJumpError)
prog.rb:6:in `block (2 levels) in <main>': unexpected return (LocalJumpError)
```

This is also true if you create the raw proc using Proc.new.

```
def meth4
  p = Proc.new { return 99 }
```